

lsmeasr: a variable selection strategy for interval branch and bound solvers

Ignacio Araya¹  · Bertrand Neveu²

Received: 30 January 2017 / Accepted: 6 September 2017
© Springer Science+Business Media, LLC 2017

Abstract Smear-based variable selection strategies are well-known and commonly used by branch-and-prune interval-based solvers. They estimate the impact of the variables on each constraint of the system by using the partial derivatives and the sizes of the variable domains. Then they aggregate these values, in some way, to estimate the impact of each variable on the whole system. The variable with the greatest impact is then selected. A problem of these strategies is that they, generally, consider all constraints equally important. In this work, we propose a new variable selection strategy which first weights the constraints by using the optimal Lagrangian multipliers of a linearization of the original problem. Then, the impact of the variables is computed with a typical smear-based function but taking into account the weights of the constraints. The strategy is tested on a set of well-known benchmark instances outperforming significantly the classical variable selection strategies.

Keywords Branch and bound · Interval-based solver · Variable selection · Lagrangian multipliers

1 Introduction

This paper deals with continuous global optimization (nonlinear programming) deterministically handled by interval branch and bound (B&B). The problem is defined by:

$$\min_{x \in \mathcal{X}} f(x) \text{ s.t. } g(x) \leq 0 \quad (1)$$

✉ Ignacio Araya
ignacio.araya@pucv.cl

Bertrand Neveu
Bertrand.Neveu@enpc.fr

¹ Pontificia Universidad Católica de Valparaíso, Escuela de Ing. Informática, 2950 Avenida Brasil, V región, Chile

² Imagine LIGM Université Paris–Est, Paris, France

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the real-valued objective (non convex) function and $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a vector-valued (non convex) function.¹ $x = (x_1, \dots, x_i, \dots, x_n)$ is a vector of variables varying in a domain (i.e., a box) $\mathbf{x}$.²

Several works have been proposed for finding good branching strategies [1–10]. Most of them use local information (provided by the current node) in order to select the next variable to bisect. Bisection consists of splitting the current box into two boxes generating two subproblems to solve. *round-robin* is one of the simplest methods. It does not require any information about the system. If the current n -dimensional box $\mathbf{x}$ was obtained by bisecting the interval x_i , then the *round-robin* method will select the variable $x_{i'}$, with $i' = (i + 1) \bmod n$. The objective is to refrain from neglecting any variable. One weakness of this strategy is that a *bad* initial ordering of variables can lead to disastrous performance. *largest-first* [8] (also known as the interval width-oriented rule [2]) simply selects the variable with the largest domain. It is based on the assumption that intervals with large diameters have a greater impact on the function image. In the same manner as the *round-robin* technique, the *largest-first* method does not omit any variable. Smear-based methods [4, 10] use information about the system to obtain the variable with the greatest *impact*. The impact of a variable x_i on a function g_j is computed by means of the *smear value*. Consider that the current node is associated with box $\mathbf{x}$; the smear value is given by

$$\text{smear}(x_i, g_j) = |a_{ji}| * \text{wid}(x_i)$$

where a_{ji} is an interval overestimate of the range of the partial derivative $\frac{\partial g_j}{\partial x_i}$ in $\mathbf{x}$.³ $|a_{ji}|$ is the *magnitude* of the interval a_{ji} , i.e., $|a_{ji}| = \max(|a_{ji}|, |\overline{a_{ji}}|)$.

Theoretically, the smear value of a variable x_i related to a function g_j estimates (using a first-order Taylor approximation of g_j) the reduction of the image size of g_j consequent upon a reduction of the domain size of x_i [1].

Selection methods based on the smear value select the variable that maximizes an aggregation of this value in the whole system. For instance, the strategy *smearmax* [10] selects the variable x_i which maximizes $\max_{j=1..m}(\text{smear}(x_i, g_j))$. The strategy *smearsum* selects the variable x_i which maximizes $\sum_{j=1}^m(\text{smear}(x_i, g_j))$. A variant proposed in [1, 6] uses a *relative smear value* instead:

$$\text{smear}_{rel}(x_i, g_j) = \frac{\text{smear}(x_i, g_j)}{\sum_{k=1}^n \text{smear}(x_k, g_j)}$$

Then, the *smearsumrel* strategy selects the variable x_i , which maximizes $\sum_{j=1}^n(\text{smear}_{rel}(x_i, g_j))$. Generally, smear-based strategies also include the objective function in the vector g for computing the impact of variables.

Tawarmalani and Sahinidis [11] present an algorithm called *ViolationTransfer*, to estimate the impact of a variable on the problem. ViolationTransfer works with the Lagrangian function of a relaxation of the problem and an optimal solution of the relaxation x^* . For each variable, an interval $\mathbf{x}_i^v \subset \mathbf{x}_i$ is defined. $\mathbf{x}^v$ is the smallest box such that it contains x^* and each univariate constraint $g_j(x_1^*, \dots, x_{i-1}^*, x_i, x_{i+1}^*, \dots, x_n^*) \leq 0$ ($j = 1..m$) is feasible for at least one value in $\mathbf{x}_i^v$.

¹ The branching strategies proposed in this paper can also apply to problems having equality constraints.

² An interval $\mathbf{x}_i = [\underline{x}_i, \overline{x}_i]$ defines the set of reals x_i s.t. $\underline{x}_i \leq x_i \leq \overline{x}_i$. A *box* $\mathbf{x}$ is a Cartesian product of intervals $\mathbf{x}_1 \times \dots \times \mathbf{x}_i \times \dots \times \mathbf{x}_n$.

³ The range of partial derivatives can be computed by using an automatic differentiation method [10].

Then, for each variable x_i , the difference between the bounds of the image of the Lagrangian function over the interval $\mathbf{x}_i^v$ is estimated. In each estimation all the variables are fixed except x_i . The assumption is that branching on the variable maximizing the image width is likely to improve the lower bound of the objective function in the subproblems. If we consider a linear relaxation: $\min \sum_{i=1}^n c_i x_i$, subject to $\sum_{i=1}^n a_{ji} x_i \leq 0$ for all $j \in \{1, \dots, m'\}$, an estimate of the change of the Lagrangian function related to the variable x_i is given by:

$$(\bar{x}_i^v - \underline{x}_i^v) \cdot \left| c_i + \sum_{j=1}^{m'} \lambda_j^* a_{ji} \right|$$

where λ^* is the *optimal dual solution* to the linear relaxation. ViolationTransfer selects the variable maximizing this value.

In this article we propose *lsmeasr*, a new variable selection strategy for interval B&B solvers. In a few words, the method selects the variable maximizing the smear value of the Lagrangian function of the problem. In the Lagrangian function, the Lagrange multipliers are replaced by the dual optimal of a linear approximation of the problem. Related to the ViolationTransfer strategy our approach has some important differences:

1. *lsmeasr* uses a simple *linear approximation* of the original problem instead of sophisticated convex relaxation techniques.
2. *lsmeasr* estimates the impact of each variable in the Lagrangian function of the *original problem*. Instead, ViolationTransfer estimates the impact of each variable in the Lagrangian function of a *linear relaxation*.
3. The computation of $\mathbf{x}^v$ requires the solver to use a reformulated problem in which multidimensional functions are replaced with either univariate or bilinear functions [11]. For the moment, and in order to maintain the simplicity and generality of the approach, *lsmeasr* uses directly $\mathbf{x}$ instead of $\mathbf{x}^v$.

In Sect. 2 we present the necessary background. Section 3 gives a brief introduction about the Lagrange function and the optimal Lagrange multipliers. In Sect. 4 we explain our approach and some variants. Section 5 carries out an empirical study of different variants of the proposed strategy. Finally, conclusions are given in Sect. 6.

2 Background and the interval solver

An **interval** $\mathbf{x}_i = [\underline{x}_i, \bar{x}_i]$ defines the set of reals x_i s.t. $\underline{x}_i \leq x_i \leq \bar{x}_i$, where $\underline{x}_i$ and $\bar{x}_i$ are floating-point numbers. $\mathbb{IR}$ denotes the set of all intervals. The size or *width* of $\mathbf{x}_i$ is $\text{wid}(\mathbf{x}_i) = \bar{x}_i - \underline{x}_i$. $\text{mid}(\mathbf{x}_i)$ denotes the midpoint of $\mathbf{x}_i$. A **box** $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ determines the Cartesian product of intervals $\mathbf{x}_1 \times \dots \times \mathbf{x}_n$. The *size* of a box is $\text{wid}(\mathbf{x}) = \max_{x_i \in \mathbf{x}} \text{wid}(\mathbf{x}_i)$.

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is *factorable* if it can be computed in a finite number of simple steps, starting with model variables and real constants, using elementary unary and binary operators. Factorable functions are extended to intervals. A function $\mathbf{f} : \mathbb{IR}^n \rightarrow \mathbb{IR}$ is an **interval extension** of a real function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ if it computes an enclosure of the image of f over any box $\mathbf{x}$, i.e., $\forall \mathbf{x} \in \mathbb{IR}^n, \mathbf{f}(\mathbf{x}) \supseteq \{f(x), x \in \mathbf{x}\}$. Interval arithmetic defines the extension of the classic elementary operators (e.g., $+$, $-$, $\cdot$, $/$, power, exp, log, sin, cos,...). For instance, $[a, b] + [c, d] = [a + b, c + d]$, $[\log]([a, b]) = [\log(a), \log(b)]$, etc. The **natural extension** of a factorable function f corresponds to the mapping of f to intervals using the corresponding interval operators.

A continuous (or numerical) constrained global optimization problem (NCOP) is defined as follows.

Definition 1 (*Numerical constrained global optimization*) Consider a vector of variables $x = (x_1, \dots, x_i, \dots, x_n)$ varying in a box $\mathbf{x}$, a real-valued factorable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, and a vector-valued factorable function $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

Given the system $S = (f, g, \mathbf{x})$, the numerical constrained global optimization problem consists in finding:

$$\min_{x \in \mathbf{x}} f(x) \text{ subject to } g(x) \leq 0$$

f denotes the **objective function**; $g(x) \leq 0$ corresponds to the set of **constraints**. x is said to be **feasible** if it satisfies the constraints. x^* denotes the **optimal solution** of the problem and $f(x^*)$ corresponds to the optimal value.

An **active constraint** at a feasible point x' is a constraint $g_j(x) \leq 0$ such that $g_j(x') = 0$. An **inactive constraint** at a point x' is a constraint $g_j(x) \leq 0$ such that $g_j(x') < 0$. Active constraints restrict the domain of feasibility in neighborhoods of x' , while inactive constraints have no influence on these neighborhoods. Thus, inactive constraints at the optimal solution x^* could be discarded from the system without altering either the optimal solution or the optimal value of the problem.

The constraints defined in an NCOP are numerical. They are equations and inequalities using mathematical operators like $+$, $\cdot$, $/$, $\exp$, $\log$ and $\sin$.

The standard interval NCOP solver

Interval solvers for NCOPs generally find one **ϵ -optimal** solution, i.e., one feasible point x' such that $f(x') - f(x^*) \leq \epsilon$, where ϵ corresponds to the desired precision of the method.⁴ Algorithm 1 shows the basic structure of an interval branch and bound solver. For simplicity we consider that f , g and x are global variables. Additionally, a variable x_o , with initial domain $\mathbf{x}_o = [-\infty, +\infty]$ is included in the set of variables x and a new constraint $x_o = f(x)$ is included in the set of constraints (actually, functions $f(x) - x_o$ and $x_o - f(x)$ are included in g). In this way, x_o keeps updated the bounds of f in the box.

Starting from an initial box $\mathbf{x}$, the solver builds a search tree by following a branch & bound schema. In each iteration a box from the list L of remaining boxes is selected and processed (first line of the while-loop). Generally, the box minimizing a lower bound of the objective function is selected [12–14]. A sophisticated method called **FeasibleDiving** [15] selects the next box in two different ways. The first way, called *diving*, does a depth-first-like search until a leaf of the tree is reached. When a leaf is found, the strategy swaps and selects the box minimizing its lower bound. Then, another diving is performed and so on.

Then, nodes are treated by two methods: bisection and filtering. The *bisection* consists in splitting the current box into two (or more) sub-boxes by dividing one (or more) interval(s) in the middle.

New boxes are then treated by the *filtering* method one by one. This procedure is composed by one or several contraction methods or *contractors*. Contractors attempt to filter the boxes by eliminating inconsistent values from within their bounds without loss of solutions. If all the values in a box are filtered, an empty box is returned. The filtering procedure is also

⁴ Due to the relative precision of floating point numbers, when the optimal cost is too large a relative precision is required, i.e., $\frac{f(x') - f(x^*)}{|f(x')|} \leq \epsilon$.

responsible for computing the lower bound of the box. This is achieved by filtering the domain of the variable x_o . Thus, after filtering, a new lower bound of the objective function in the box is given by $\underline{x}_o$. Several contractors have been proposed so far, they include contractors based on constraint propagation [7, 16–19] and contractors based on linear relaxations [20–22].

```

Data:  $f, g, x$ 
procedure Solver ( $x, \epsilon$ ); out:  $x_{UB}, UB$ 
   $L \leftarrow x; UB \leftarrow +\infty; LB \leftarrow -\infty;$ 
  while  $L \neq \emptyset$  and  $UB - LB > \epsilon$  and  $\frac{UB-LB}{|UB|} > \epsilon$  do
     $x \leftarrow$  select a box from  $L$ ;
     $(x^l, x^r) \leftarrow$  bisect( $x$ );
    for  $x \in \{x^l, x^r\}$  do
       $x \leftarrow$  filtering( $x$ );
      if  $x \neq \emptyset$  then
        upper-bounding( $x, UB, x_{UB}$ );
        insert( $L, x$ );
      end
    end
     $LB \leftarrow \min_{x \in L} \underline{x}_o$ ;
  end
end.

```

Algorithm 1: Algorithm NCOP-solver

The *upper bounding* consists in finding feasible solutions in x with costs lower than the current UB . If a best solution is found, the procedure updates UB and the best found solution x_{UB} . Updating UB implies a *global* reduction of the feasible space through the constraint $f(x) \leq UB$. This constraint is considered by the filtering procedure. Before filtering, the procedure updates the upper bound of the variable x_o in the box: $x_o \leftarrow x_o \cap [-\infty, UB]$. To find feasible points, generally local search methods are used (e.g., selecting the midpoint of the box [13], applying the Newton method [12, 23], extracting convex inner regions [14]), plus some feasibility tests (e.g., the natural evaluation of the candidate point [13, 14], the Borsuk proof [12], etc.). Then, x inserted into the list L .

LB is the current computed lower bound of the objective function. It corresponds to the minimum value of the lower bounds for all the leaf boxes in the tree. The search concludes when one of the following termination criteria is reached:

- If $UB - LB \leq \epsilon$ or $\frac{UB-LB}{|UB|} \leq \epsilon$, the solver returns an ϵ -optimum solution of the problem x_{UB} and its cost UB . Note that if some boxes still remain in L , the lower bound of these boxes should necessarily be equal or greater than $UB - \epsilon$ or $UB - |UB|\epsilon$.
- If $L = \emptyset$ and no feasible solution was found, then the problem is unfeasible.

3 Optimal Lagrange multipliers

In this section we give an introduction to the *Lagrangian* and the so-called *optimal Lagrange multipliers* which will be used by our approach to weight the impact of constraints in the optimization problems.

The Lagrangian $L : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ associated to an optimization problem $\min_{x \in \mathbf{x}} f(x)$ s.t. $g(x) \leq 0$ (or *primal problem*) is defined by:

$$L(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x)$$

where λ_j is the *Lagrange multiplier* associated to the j -th inequality constraint $g_j(x) \leq 0$.

The Lagrangian basically augments the objective function with a weighted sum of the constraint functions. The *Lagrange dual function* $h : \mathbb{R}^m \rightarrow \mathbb{R}$ is defined by:

$$h(\lambda) = \inf_{x \in \mathbf{x}} L(x, \lambda)$$

An important property of the dual function is that for any value of $\lambda \geq 0$: $h(\lambda) \leq f(x^*)$. Finally, the *Lagrange dual problem* associated with the problem (1) is defined by:

$$\max_{\lambda} h(\lambda) \text{ subject to } \lambda \geq 0$$

λ^* denotes the dual optimal solution of the problem and its scalar components are called **optimal Lagrange multipliers**. As seen from function L , the Lagrange multipliers are associated to each constraint of the system. Thus, in a way, the optimal Lagrange multiplier related to any constraint estimates the rate of change in the optimal value, consequent upon changes in the constraint function. In particular, the value of the optimal Lagrange multipliers is 0 for inactive constraints of the problem.

Consider the following linear program in inequality form, i.e.,

$$\min_x c^T x \text{ subject to } Ax \leq b$$

where A is the coefficient matrix, b and c are vectors of coefficients and c^T is the transpose of vector c . The dual problem associated to the primal one can be formulated in the following way:

$$\max_{\lambda} -b^T \lambda \text{ subject to } A^T \lambda + c = 0, \lambda \geq 0$$

4 lsmeasr, a smear-based strategy using optimal Lagrange multipliers

A main issue related to the smear-based strategies is that these strategies do not weight the importance of each constraint, i.e., they consider all the constraints equally important (even between active and inactive constraints).

On the other hand, as noted in the previous section, if constraints of the Lagrangian function of the problem are weighted by means of the optimal Lagrange multipliers, the problem of minimizing the Lagrangian function can be seen as an approximation of the original optimization problem.

Thus, lsmeasr, our proposed algorithm, is basically a smear-based strategy which uses an *estimation* of the optimal Lagrange multipliers for weighting the constraints. The algorithm works in two phases: first, a linearization of the global optimization problem is generated. Each function $g_j(x)$ is approximated by using the first order term of its Taylor expansion around the midpoint of the box, i.e.,

$$gl_j(x) = g_j(\text{mid}(\mathbf{x})) + \sum_{i=1}^n \text{mid}(J_{ji}) \cdot (x_i - \text{mid}(x_i))$$

where J_{ji} is an interval overestimation of the image of $\frac{\partial g_j}{\partial x_i}$ over x computed by using automatic differentiation [10]. Note that instead of using the partial derivatives in the midpoint of the box we use the midpoint of the overestimation of the partial derivatives (i.e., $\text{mid}(J_{ji})$).

The generated linear optimization problem includes the bound constraints, i.e., $x_i \leq \bar{x}_i$, and it is solved by using the simplex method.⁵ If the linear program does not have solutions or if the optimal value is unbounded, then the `smeasum` strategy simply selects the next variable to bisection. Otherwise, if an optimum exists, then a second phase is carried out. In this phase, the strategy computes the smear values of the variables on the following function:

$$L(x) = x_o + \sum_{i=1}^m \lambda_j^* g_j(x)$$

where λ^* corresponds to the dual optimal solution of the linear problem. The function L is equivalent to the Lagrangian of the original problem but the Lagrange multipliers have been replaced by the optimal Lagrange multipliers of the linear approximation. Finally, `lsmeas` selects and bisection the variable that maximizes its smear value.

An interesting thing about L is that it approximates the original problem by only one objective function. When there is no constraint, there is no disagreement between different smear-based methods either: they all should select the same variable that maximizes the smear value.

```

procedure lsmeas( $x, J, g$ ); out: var
  /* Phase 1: linearization and solving the linear program */;
   $gl \leftarrow g(\text{mid}(x)) + \text{mid}(J) \cdot (x - \text{mid}(x))$ ;
   $t \leftarrow x_o$ ;  $x_o \leftarrow [-\infty, \infty]$ ;
   $\lambda^* \leftarrow \text{optimize}(\min x_o, \text{ subject to: } x_i \leq \bar{x}_i, gl_j(x) \leq 0)$ ;
   $x_o \leftarrow t$ ;
  if  $\lambda^* \neq \emptyset$  then
    /* Phase 2: computing the impact of  $L(x)$  */;
     $\text{max\_impact} \leftarrow 0$ ;
    for  $i \in \{1..n\}$  do
       $D \leftarrow \lambda_i^*$ ;
      for  $j \in \{1..m\}$  do
         $D \leftarrow D + \lambda_{n+j}^* \cdot J_{ji}$ ;
      end
      if  $|D.\text{wid}(x_i)| > \text{max\_impact}$  then
         $\text{max\_impact} \leftarrow |D.\text{wid}(x_i)|$ ;
         $\text{var} \leftarrow i$ ;
      end
    end
    return  $\text{var}$ ;
  else
    return smeasum( $x, J$ );
  end
end.

```

Algorithm 2: `lsmeas`

⁵ Actually we need to solve the dual problem, however we use a linear solver (SoPlex [24]) which solves both, the primal and the dual problems

Algorithm 2 shows our approach. $\mathbf{J}$ corresponds to the Jacobian matrix which contains the interval overestimations of the partial derivatives over $\mathbf{x}$. In the linear program (Phase 1), the interval related to the objective variable is unbounded to enhance the chances for successfully finding an optimal solution. In Phase 2, for each variable x_i we first compute $\mathbf{D}$, which is an interval overestimation of $\frac{\partial L}{\partial x_i}$ over $\mathbf{x}$. The overestimation is obtained by adding the products of the interval partial derivatives on each constraint ($\mathbf{J}_{ji}$) and the corresponding dual optimal value (λ_{n+j}^*). $\mathbf{D}$ is initialized with the dual optimal value related to the bounded constraint, i.e., λ_i^* (the partial derivative related to the i -th bound constraint over the variable x_i is 1). Then, the smear impact of the variable corresponds to the magnitude of the product of the interval partial derivative and the width of the related interval, i.e., $|\mathbf{D}.wid(x_i)|$. The variable with the maximum impact is saved and returned. If the linear program does not have solutions or if the optimal value is unbounded, then the `smearsum` method is launched instead. This method uses the same Jacobian matrix used by `lsmeasr`.

4.1 Variants

We implemented four variants of the basic strategy. The first of them, `lsmeasr-MG` (Mid-point Gradient) generates the linearization by using the gradient in the midpoint of the box, instead of using the midpoint of the *overestimate* of the partial derivatives in the box. Thus, `lsmeasr-MG` requires to compute two gradients, one for generating the linearization, and the other for computing the smear impact of the composed function.

A second variant is called `lsmeasr-LI` (Linear Impact). It computes the impact using the Lagrangian of the linearized problem, i.e.,

$$L'(x) = x_o + \sum_{i=1}^m \lambda_j^* g l_j(x)$$

To do so, we only need to replace $\mathbf{J}_{ji}$, by $\text{mid}(\mathbf{J}_{ji})$ in Phase 2. $\text{mid}(\mathbf{J}_{ji})$ corresponds to the partial derivative $\frac{\partial g l_j}{\partial x_i}$.

`lsmeasr-LR` (Linear Relaxation), uses a linear relaxation of the system generated by the linear-relaxation-based contractor of Ibex. This contractor generates a relaxed linear program by relaxing each constraint using two methods: X-Taylor and AF2. After the relaxation, each nonlinear constraint is represented by, between 0 to several, linear constraints in the linear program. In order to compute the vector λ^* which will be used to weight the constraints for computing the variable impacts, we have to map the dual optimal values related to the linear program to the λ^* vector related to the original nonlinear constraints. Thus, for instance, if a nonlinear constraint g_i was approximated by three linear constraints, then λ_i^* is computed by adding the dual optimal values related to these three constraints. Furthermore, for implementing `lsmeasr-LR` we have to modify slightly the solver algorithm 1. Because we need that the linear-relaxation-based contractor computes λ^* , and this contractor is inside the pruning method, we compute and choose the next variable to bisection, related to a box $\mathbf{x}$, just after pruning this box (without even bisecting $\mathbf{x}$). The chosen variable is stored in the corresponding node until the node is treated again and a bisection on this variable domain is performed.

`lsmeasr-LRI` (Linear Relaxation Impact) uses the same linear relaxation used by `lsmeasr-LR` for computing the vector λ^* . Then, it computes the impact using the Lagrangian of the linear relaxation instead of the Lagrangian of the original problem. This variant follows the same idea as ViolationTransfer but using a different convex relaxation and the box $\mathbf{x}$ instead of $\mathbf{x}^v$.

Remark that, the implementation of `lsmeas-LR`, `lsmeas-LRI` and `ViolationTransfer` require to modify the convex-relaxation-based contractor of the solver in order to obtain the dual optimal values. That is not the case of strategies `lsmeas`, `lsmeas-LI` and `lsmeas-MG` which use a cheap and independent linearization-based method for computing these values.

5 Experiments

In order to validate our approach, we implemented the `lsmeas` method and its variants in `IbexOpt`, a state-of-the-art optimizer of the Interval-Based EXplorer library (`Ibex` [25]).

`IbexOpt` is composed of several contractors including: `HC4` [16], `ACID(HC4)` [19] and a linear relaxation based contractor combining two relaxation methods: `AF2` [13] and `XNewton` [22]. For the upper bounding `IbexOpt` uses `inHC4` and `InnerPolytope` [14]. The selection of the next box is performed by `FeasibleDiving` [15]. Precision parameter ϵ was set to 10^{-6} (value commonly used by other solver benchmarks, e.g., [26,27]).

All the experiments were run on a server PowerEdge T420, with 2 quad-processors Intel Xeon, 2.20 GHz and 8 GB RAM.

The instances were selected from the series 1 and 2 of the COCONUT constrained global optimization benchmarks.⁶ We selected all the problems solved by some strategy in a time comprised between 2s and 7200s (76 instances). Due to some random choices performed inside `Ibex`, each strategy was run five times on each instance. In order to avoid outliers we removed the lowest and highest CPU time and we report the average of the remaining three executions. Timeouts indicate that the strategy could not solve the instance in less than 7200 seconds in any of the three executions.

Definition 2 Average relative gain. We define as relative time $t_r(a, b, \pi)$ the ratio between the mean time ($\bar{t}$) taken by a strategy a and the sum of mean times taken by the strategies a and b in solving an instance π , i.e., $t_r(a, b, \pi) = \frac{\bar{t}(a, \pi)}{\bar{t}(a, \pi) + \bar{t}(b, \pi)}$. Thus, the **average relative gain** of a strategy a w.r.t. a strategy b is given by $\frac{\sum_{\pi \in \Pi} (t_r(b, a, \pi))}{\sum_{\pi \in \Pi} (t_r(a, b, \pi))}$, where Π is the set of the considered instances.

5.1 `lsmeas` v/s classical strategies

In a first series of experiments, we compared the proposed strategy `lsmeas` with some of the classical variable selection strategies: `round-robin` (`rr`), `largest-first` (`lf`), `smearsum` (`ssum`), `smearmax` (`smax`) and `smearsumrel` (`ssr`). All strategies are restricted to select variables with interval sizes larger than 10^{-7} . This condition avoids, in some pathological cases, to bisect some variables to almost null-size intervals.⁷

Figure 1 summarizes the comparison among the six strategies. From the set of instances solved in [2, 7200] seconds by at least one of the six strategies, each curve reports the proportion of instances solved by the corresponding strategy in less than or equal to *factor* times the best reported CPU time. Factor corresponds to the CPU time spent by the strategy divided by the best CPU time spent on each instance.

⁶ www.mat.univie.ac.at/~neum/glopt/coconut/Benchmark/Benchmark.html.

⁷ When a box reaches a size lower than 10^{-7} , the bisector selects the next variable without taking into account the restriction. This situation is very rare and does not occur in any of the selected instances.

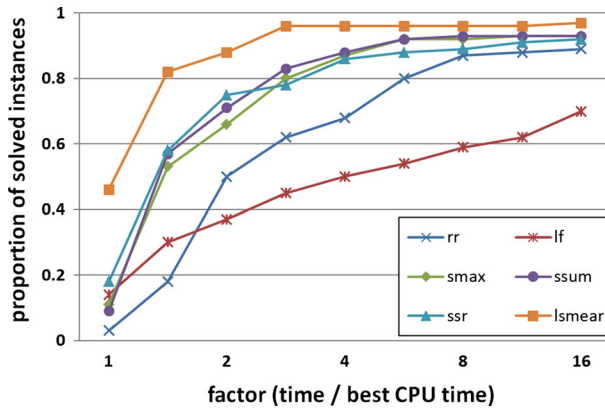


Fig. 1 Performance profile. Comparison among *lsmeasr* and the classical variable selection strategies

From the figure we observe that *lf* reports the worst results. The *smear*-based strategies: *ssum*, *smax* and *ssr* report comparable performances. *lsmeasr* clearly outperforms all the classical variable selection strategies. Note, for instance, that almost 90% of the instances are solved by *lsmeasr* in less than twice the best CPU time reported by all the strategies.

Table 1 reports the results of the strategies for a subset of instances. The subset comprises all the instances such that the ratio between the lowest and largest CPU time among the strategies is larger than or equal to 5.0. For each strategy we report the CPU time and the size of the tree search in number of treated nodes. The last column reports, for each instance, the gain of *lsmeasr* w.r.t. the best classical strategy on that particular instance. The gain is computed: $best_time/time(lsmeasr)$. The last row reports the CPU time spent by each strategy in the whole set of 76 instances. Timeouts are represented by the letters “to”. A superindex i in the CPU time indicates that i of the 3 executions of the corresponding strategy reached a timeout. In this case, the CPU time and the number of nodes are computed with the executions completed in less than 7200 seconds.

Comparing the times and number of boxes for different strategies, remark that solving the linear program by the *lsmeasr* bisection seems to produce a negligible run-time overhead. This is mainly due to the contractor of the solver, based on X-Taylor and AF2, is much more expensive: it requires to solve up to $2n$ linear programs in each node for attempting to improve the bounds of the n variable domains.

Figure 2 compares the CPU times among *lsmeasr* and the classical strategies. The average relative gain of *lsmeasr* w.r.t. the other strategies is 2.04 (*rr*), 2.48 (*lf*), 1.34 (*smax*), 1.30 (*ssum*) and 1.32 (*ssr*). We then consider that *ssum* is the best classical strategy and will be used as reference in the next sections.

5.2 Variants of *lsmeasr*

In a second series of experiments we compared the different proposed variants of *lsmeasr*. Figure 3 summarizes the comparison among the strategies.

ssum-active corresponds to a variant of *smearsum* in which *some* inactive constraints are detected and discarded. We discard those constraints $g_j(x) \leq 0$ such that the enclosure of the image, computed by means of the natural extension, of g_j over the current box is strictly lower than 0. From the set of instances solved in $[2, 7200]$ seconds by at least one of

Table 1 Average CPU time and number of boxes for the strategies rr, lf, ssum, smax and ssr and lsmeasr in a subset of instances

Instance	rr		lf		ssum		smax		ssr		lsmeasr		gain
	time	#box	time	#box	time	#box	time	#box	time	#box	time	#box	
ex5_4_4	1020	12,626	to	to	200	2626	222	3248	330	4148	229	2773	0.87
ex6_1_1	22	3002	86	20,788	13	1505	13	1625	13	1590	13	1615	1.00
ex6_1_3	227	12,109	899	85,173	63	3215	59	3163	73	3516	54	2894	1.09
ex6_2_6	156	88,643	3577	2,581,127	28	14,583	28	14,565	32	16,621	25	12,711	1.13
ex6_2_8	67	36,964	1629	952,892	19	9535	19	9376	19	9439	16	8456	1.18
ex6_2_9	50	25,153	2702	2,136,817	15	6157	13	5553	11	4816	11	4796	1.00
ex6_2_10	3191	1,150,155	to	to	268	68,736	269	68,996	269	68,993	275	68,746	0.97
ex6_2_11	31	19,959	564	365,123	11	5932	11	5823	11	6259	10	5191	1.10
ex6_2_12	13	7529	236	233,560	6.0	2681	5.2	2396	4.9	2276	3.9	1833	1.26
ex7_2_3	7.2	1703	23	7123	17	5380	20	7029	8.0	2075	2753	1,877,486	0.00
ex7_2_9	20	3090	18	2614	14	1720	14	1686	to	to	34	11639	0.39
ex7_3_4	to	2,615,244	to	to	14	1609	20	2035	4.8	462	5.3	588	0.91
ex7_3_5	5.7	295	19	917	7.9	364	8.0	388	3.9	247	4.2	250	0.93
ex8_4_4	263	5894	384	10,239	38	867	61	1476	41	959	61	1566	0.61
ex8_4_5	2258	87,265	39	1127	210	8724	220	9681	221	10,292	612	73,127	0.06
ex8_5_1	6.5	1744	to	to	5.3	1157	4.7	1024	5.8	1279	4.1	888	1.15
ex8_5_2	14	3267	to	to	15	3109	12	2542	17	3415	10	2126	1.26
ex14_2_1	1.0	270	7.0	2209	0.5	77	0.4	63	0.5	70	0.4	61	1.00
ex14_2_3	2.0	284	28	4573	2.0	259	2.0	255	2.0	251	1.7	207	1.18
ex14_2_7	52	7567	1553	223,378	23	2099	24	2252	22	2087	25	2209	0.91
bearing	27	9610	114	39,701	44	11,798	48	13,251	5.6	690	15	1713	0.38
chem	3734	520,476	to	to	7173 ⁽²⁾	838,170	to	to	6513	803,920	393	50,555	9.16
harker	1400	69,031	to	to	3035	138,881	2410	113,515	to	to	1071	50555	1.31
immun	3.7	661	16	1949	19	2552	16	2336	31	4670	9.3	1606	0.40

Table 1 continued

Instance	rr		lf		ssum		smax		ssr		lsmeat		gain
	time	#box	time	#box	time	#box	time	#box	time	#box	time	#box	
launch	114	2358	96	1501	47	772	46	733	46	763	15	214	3.01
linear	to	to	56	1112	214	5511	120	3501	226	5857	137	3811	0.41
process	1.9	319	14	1822	2.2	339	1.6	233	1.3	194	1.7	233	0.76
ramsey	18	361	10	182	10	160	11	189	to	to	11	207	0.89
srcpm	to	to	to	to	487	19,515	331	13,985	to	to	97	3559	3.43
batch	to	30,392	to	to	105	863	105	843	98	733	54	432	1.83
disc2	to	to	to	to	to	to	to	to	33	673	32	502	1.03
dualc1	11	1588	3050	306,956	5.1	485	5.2	474	5.0	477	5.0	494	1.00
dualc2	1.0	160	1263	161,200	0.8	107	0.8	101	0.9	107	0.8	93	1.00
dualc5	69	17,525	to	to	53	12,604	53	12,616	54	11,992	58	14,301	0.91
haifas	269	98,827	to	to	4532	1,607,782	to	to	2.0	890	1.7	722	1.18
haldmads	to	to	39	4390	6271	232,187	5973	263,801	6500	294,930	5189	216,019	0.01
hs087	2.3	392	8.9	1448	10	1543	28	4061	1.0	145	0.6	65	1.67
hs093	3102	1,735,441	to	to	670	227,936	654	219,361	658	222,024	744	254,136	0.88
hs103	1068	172,019	to	to	726	95,312	790	108,327	590	75179	623	76,421	0.95
hs117	1981	146,455	to	to	383	20,215	581	30,717	520	28,723	272	14,503	1.41
hs119	20	576	187	6049	14	381	13	374	18	448	14	428	0.94
makela3	3.5	885	2.8	368	0.9	149	4.9	1144	0.9	149	1.2	193	0.75
mistake	1063	141,163	to	to	1077	136,450	601	71,593	to	to	732	81,966	0.82
odfirs	66	16,905	92	27,451	19	3513	17	2792	to	to	15	2561	1.13
robot	1725	447,608	343	74,593	674	153,963	922	220,925	573	128,116	367	79,535	0.94
total time	77,602		144,472		43,867		47,474		65,789		21,035		2.09

In bold, the best CPU times and the smallest number of boxes reported on each instance

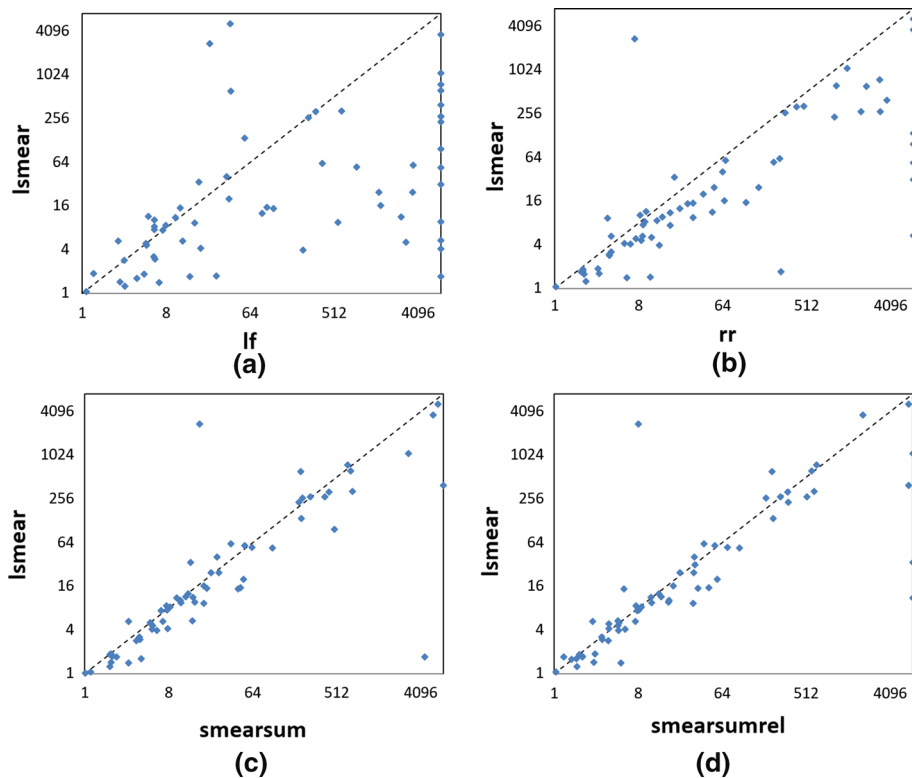


Fig. 2 Each plot compares the CPU times of two strategies for each of the instances. Each point corresponds to an instance and its coordinates indicate the average CPU time (in seconds) required by each of the two strategies in solving this instance. For example, the bottom right point in the *lsmeat* versus *smearsum* plot corresponds to an instance solved in about 2s by the *lsmeat* strategy and about 4096s by *smearsum*. Note that most of the points appear in the lower-right side of the figure, which highlights that *lsmeat* dominates the other strategies on most instances. **a** *lsmeat* versus largest-first. **b** *lsmeat* versus round-robin. **c** *lsmeat* versus *smearsum*. **d** *lsmeat* versus *smearsumrel*

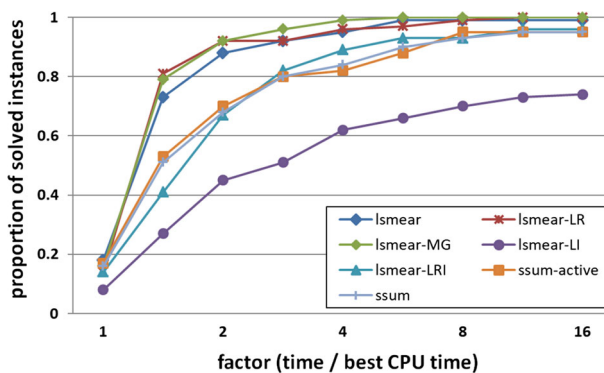


Fig. 3 Performance profile. Comparison between the different variants of *lsmeat*

Table 2 Average CPU time and gain for all the strategies in a subset of instances. In bold, the best CPU times and the smallest number of boxes reported on each instance

Instance	lsmeasr			lsmeasr-MG			lsmeasr-LR		
	time	#box	gain	time	#box	gain	time	#box	gain
hs087	0.6	65	1.00	0.6	46	1.00	5.7	953	0.11
bearing	15	1713	1.05	16	1877	0.96	57	14,197	0.26
harker	1071	50,555	1.41	1514	81,121	0.71	3865	203,997	0.28
immun	9.3	1606	0.54	8.6	1660	0.58	5.0	829	1.72
linear	137	3811	0.19	25	1557	4.53	115	3360	0.22
ex7_2_3	2753	1,877,486	0.00	11	3557	1.52	16	4846	0.66
srcpm	97	3559	1.13	110	3552	0.88	305	12573	0.32
optprloc	2.8	9	0.36	2.8	9	0.36	1.0	11	2.80
ex7_3_4	5.3	588	0.83	4.4	403	1.20	11	1268	0.41
ex8_2_1	4.6	10	0.26	4.7	11	0.26	1.2	12	3.83
ex8_4_5	612	73,127	0.35	647	548,282	0.33	211	8826	2.90
hs114	1.5	190	0.60	1.7	268	0.53	0.9	122	1.67
total time	21,035		0.91	19,181		1.07	20,429		0.94

the six strategies, each curve reports the proportion of instances solved by the corresponding strategy in less than *factor* times the best reported CPU time.

Note that lsmeasr-MG, lsmeasr-LR and lsmeasr show the best performance. The average relative gain of the best variant, lsmeasr-MG, w.r.t. the others is 1.07 (lsmeasr), 1.05 (lsmeasr-LR), 2.25 (lsmeasr-LI), 1.43 (lsmeasr-LRI), 1.42 (ssum-active) and 1.43 (ssum). The variants lsmeasr-LRI and lsmeasr-LI, which compute the impact using the Lagrangian of the linear relaxation, instead of the Lagrangian of the original problem, report the worst results.

ssum-active reports results quite similar to those of smearsum, highlighting that only detecting active constraints but without pondering them, does not improve the performance of the smear-based strategies.

Table 2 reports the results of lsmeasr and its best variants for a subset of instances. The subset comprises all the instances such that the ratio between the lowest and largest CPU time among the strategies is larger or equal than 2.0. For each strategy we report the CPU time, the number of boxes of the search tree and the gain (i.e., the quotient between the best CPU time reported by the other strategies and the CPU time reported by the corresponding strategy). The last row reports the CPU time spent by each strategy in the whole set of instances.

Figure 4-left compares the CPU times among lsmeasr and lsmeasr-MG. Note that, excepting two or three instances in which lsmeasr-MG reports the best performance, the results are quite similar. Figure 4-right compares the CPU times among lsmeasr-LR and lsmeasr-MG. Both strategies report comparable results in the set of instances. However, we prefer lsmeasr-MG, because (1) according to the average relative gain (1.05) it is slightly better than lsmeasr-LR and (2) its implementation is simpler: it does not require to modify the contractor of the solver.

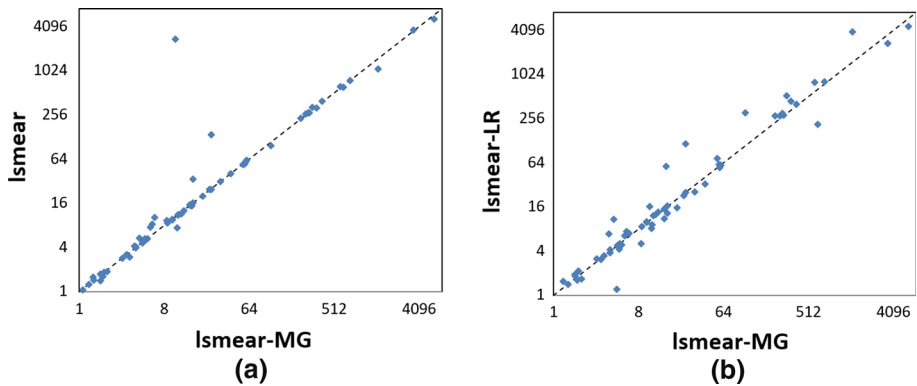


Fig. 4 Comparison between `lsmeas-MG`, `lsmeas` and `lsmeas-LR`. Each plot compares the CPU times of two strategies for each of the instances. Each point corresponds to an instance and its coordinates indicate the average CPU time (in seconds) required by each of the two strategies in solving this instance. **a** `lsmeas` versus `lsmeas-MG`. **b** `lsmeas` versus `lsmeas-LR`

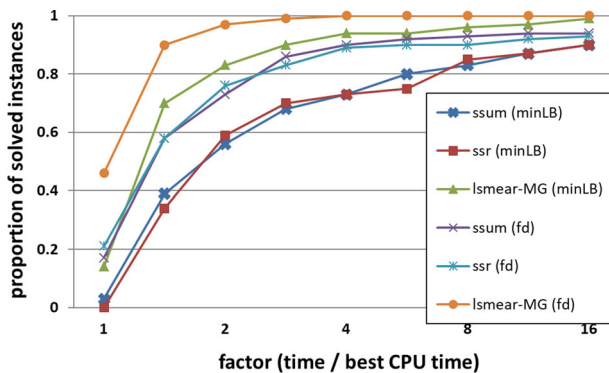


Fig. 5 Performance profile. Comparison of variable selection strategies using different search strategies

5.3 Evaluating `lsmeas` with different search strategies

In this section we evaluate the performance of `lsmeas` when two different search strategies are used: `FeasibleDiving` and `minLB`. `minLB` is a commonly used search strategy which selects next the box minimizing the lower bound of the objective function [12–14]. In [15] we showed that `FeasibleDiving` outperforms significantly `minLB` on interval-based solvers.

Figure 5 summarizes the comparison among the strategies `smearsum`, `smearsumrel` and `lsmeas-MG` using `minLB` and the `FeasibleDiving` (`fd`) search strategy.

From the set of instances solved in [2, 7200] seconds by at least one of the strategies, each curve reports the proportion of instances solved by the corresponding strategy in less than *factor* times the best reported CPU time. First note that the `FeasibleDiving` strategy outperforms the classical `minLB` approach no matter which variable selection strategy is used. Similarly, the `lsmeas-MG` strategy outperforms the `smearsum` and `smearsumrel` methods no matter which node selection strategy is used. Furthermore, note that the performance of `lsmeas-MG` using the `minLB` strategy is slightly better than `smearsum` and `smearsumrel` using the `FeasibleDiving` search strategy. The average relative gain of

Table 3 Average CPU time for the strategies in a subset of instances. In bold, the best CPU time reported on each instance

	minLB				FeasibleDiving			
	ssum	ssr	MG	gain	ssum	ssr	MG	gain
ex7_2_9	20	to	34	0.59	14	to	16	0.85
ex7_3_4	32	4.9	6.9	0.71	14	4.8	4.4	1.11
ex8_4_5	to	to	to	1.00	210	221	647	0.32
ex14_1_7	924	285	296	0.97	221	188	254	0.74
alkyl	9.3	4.4	0.9	4.66	1.9	2.1	0.7	2.65
bearing	28	8.0	6.3	1.27	44	5.6	16	0.36
chem	7167 ⁽²⁾	6477	397	16.3	7173 ⁽²⁾	6513	381	17.1
harker	3522	to	2082	1.69	3035	to	1514	2.00
hhfair	9.1	12	21	0.44	6.7	7.8	11	0.61
himmel16	110	163	31	3.51	26	32	41	0.65
hydro	121	174	134	0.90	20	35	15	1.37
immun	271	2887	128	2.12	19	31	8.6	2.23
launch	to	45	21	2.19	47	46	15	3.05
linear	321	335	36	9.03	214	226	25	8.43
process	5.6	3.4	1.1	2.99	2.2	1.3	0.6	2.21
ramsey	55	to	20	2.72	10	to	11	0.88
srcpm	103	to	134	0.77	487	to	110	4.44
batch	365	436	92	3.96	105	98	55	1.78
disc2	to	190	239	0.80	to	33	32	1.03
haifas	2062	3.3	3.4	0.95	4532	2.0	1.7	1.18
himmelbk	177	144	29	5.02	51	57	21	2.45
hs087	70	0.9	0.9	0.93	10	1.0	0.6	1.79
hs104	20	54	7.7	2.60	11	17	6.4	1.68
hs114	6.8	10	1.8	3.70	3	5.2	1.7	1.76
mistake	735	5002	603	1.22	1077	to	542	1.99
odfits	18	to	15	1.24	19	to	14	1.28
total time	59,274	76,392	29,136	2.03	43,867	65,789	19,181	2.29

lsmeas-MG w.r.t. smearsum when the minLB search strategy is used is 1.63. The average relative gain of lsmeas-MG w.r.t. smearsum when the FeasibleDiving search strategy is used is 1.43.

Finally, table 3 reports the results of the same strategies for a subset of instances. The subset comprises all the instances such that the ratio between the lowest and largest CPU time among the strategies is larger or equal than 3.0. For each strategy we report the CPU time. We also report the gain of lsmeas-MG w.r.t. the best competitor for each search strategy. The last row reports the CPU time spent by each strategy in the whole set of instances.

The best times are highlighted in bold. Note that most of them correspond to the lsmeas-MG strategy.

6 Conclusions

In this work we present `lsmeasr`, a new variable selection strategy for interval-based solvers. Unlike the well-known smear-based strategies, `lsmeasr` weights the constraints by using the optimal Lagrangian multipliers of a linearization of the problem and computes the smear value of a Lagrangian-like function. The Lagrangian multipliers are simply computed by solving a Taylor-based linearization of the original problem by using the simplex method.

The results are promising. `lsmeasr` and `lsmeasr-MG` (its best variant) significantly outperform all the bisection strategies used by classical interval-based solvers. They also outperform `lsmeasr-LRI` a variant closer to the method ViolationTransfer which also uses the optimal dual values. The difference in performance between `lsmeasr-MG` and `lsmeasr-LRI` is mainly due to that, after computing the dual values, `lsmeasr-MG` computes the variable impacts using the original constraints of the system instead of using a linear relaxation of them.

Related to ViolationTransfer, `lsmeasr` and `lsmeasr-MG` have another advantage: they are independent of the convex-relaxation-based contractor used by the solver. ViolationTransfer is intrusive in the sense that it requires to modify the contractor for obtaining the optimal Lagrangian multipliers. Furthermore, ViolationTransfer requires to modify the general strategy of the solver: variables to bisection are chosen just after pruning the box, thus, this election is saved in the node until the node is treated again and the bisection is actually performed.

In future work we will attempt to include the mechanism of ViolationTransfer which also uses the constraint violation errors to weight the variables. According to this strategy, variables related *only* to linear constraints would not have any impact and they should not be bisected. This make sense, since contractors based on linear relaxations are optimal for reducing these variable domains. However, it is also true that bisecting this variables could help, indirectly, to reduce the domains of other variables related to nonlinear constraints.

Acknowledgements Ignacio Araya is supported by the Fondecyt Project 1160224.

References

1. Araya, I., Reyes, V., Oreallana, C.: More smear-based variable selection heuristics for NCSPs. In: International Conference on Tools with Artificial Intelligence (ICTAI 2013). IEEE, pp. 1004–1011 (2013)
2. Csendes, T., Ratz, D.: Subdivision direction selection in interval methods for global optimization. SIAM J. Numer. Anal. **34**(3), 922–938 (1997)
3. Granvilliers, L.: Adaptive bisection of numerical CSPs. In: Principles and Practice of Constraint Programming, pp. 290–298. Springer, New York (2012)
4. Kearfott, R.B., Novoa III, M.: Algorithm 681: INTBIS, a portable interval Newton/bisection package. ACM Trans. Math. Softw. (TOMS) **16**(2), 152–157 (1990)
5. Lagouanelle, J.-L., Soubry, G.: Optimal multisections in interval branch-and-bound methods of global optimization. J. Glob. Optim. **30**(1), 23–38 (2004)
6. Trombettoni, G., Araya, I., Neveu, B., Chabert, G.: Inner regions and interval linearizations for global optimization. In: AAAI Conference on Artificial Intelligence, pp. 99–104 (2011)
7. Trombettoni, G., Chabert, G.: Constructive interval disjunction. Princ. Pract. Constr. Program. **2007**, 635–650 (2007)
8. Moore, R.: Interval analysis, vol. 60. Prentice-Hall Englewood Cliffs, New Jersey (1966)
9. Ratz, D.: Automatische ergebnisverifikation bei globalen optimierungsproblemen. Ph.D. dissertation, Universität Karlsruhe (1992)
10. Hansen, E., Walster, G.W.: Global Optimization Using Interval Analysis: Revised and Expanded, vol. 264. CRC Press, Boca Raton (2003)
11. Tawarmalani, M., Sahinidis, N.V.: Global optimization of mixed-integer nonlinear programs: a theoretical and computational study. Math. Program. **99**(3), 563–591 (2004)

12. Lebbah, Y.: Icos: a branch and bound based solver for rigorous global optimization. *Optim. Methods Softw.* **24**(4–5), 709–726 (2009)
13. Ninin, J., Messine, F., Hansen, P.: A reliable affine relaxation method for global optimization. *4OR* **13**(3), 247–277 (2015)
14. Araya, I., Trombettoni, G., Neveu, B., Chabert, G.: Upper bounding in inner regions for global optimization under inequality constraints. *J. Optim. Glob.* (2014). doi:[10.1007/s10898-014-0145-7](https://doi.org/10.1007/s10898-014-0145-7)
15. Neveu, B., Trombettoni, G., Araya, I.: Node selection strategies in interval branch and bound algorithms. *J. Glob. Optim.* **64**(2), 289–304 (2016)
16. Benhamou, F., Goualard, F., Granvilliers, L., Puget, J.-F.: Revising hull and box consistency. In: *International Conference on Logic Programming*, Citeseer (1999)
17. Araya, I., Trombettoni, G., Neveu, B., et al.: Exploiting monotonicity in interval constraint propagation. In: *AAAI* (2010)
18. Lhomme, O.: Consistency techniques for numeric CSPs. In: *IJCAI*. Citeseer, pp. 232–238 (1993)
19. Neveu, B., Trombettoni, G., et al.: Adaptive constructive interval disjunction. In: *International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 900–906 (2013)
20. Lebbah, Y., Michel, C., Rueher, M.: An efficient and safe framework for solving optimization problems. *J. Comput. Appl. Math.* **199**(2), 372–377 (2007)
21. Baharev, A., Achterberg, T., Rév, E.: Computation of an extractive distillation column with affine arithmetic. *AIChE J.* **55**(7), 1695–1704 (2009)
22. Araya, I., Trombettoni, G., Neveu, B.: A contractor based on convex interval taylor. In: *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pp. 1–16. Springer, New York (2012)
23. Goldsztejn, A., Lebbah, Y., Michel, C., Rueher, M.: Revisiting the upper bounding process in a safe branch and bound algorithm. In: *Principles and Practice of Constraint Programming (CP)*, pp. 598–602. Springer, New York (2008)
24. Wunderling, R.: Soplex: The sequential object-oriented simplex class library. <http://www.zib.de/Optimization/Software/Soplex/soplex.php> (1997)
25. Chabert, G., Jaulin, L.: Contractor programming. *Artif. Intell.* **173**, 1079–1100 (2009)
26. Tawarmalani, M., Sahinidis, N.V.: A polyhedral branch-and-cut approach to global optimization. *Math. Program.* **103**(2), 225–249 (2005)
27. Misener, R., Floudas, C.A.: ANTIGONE: Algorithms for continuous/integer global optimization of non-linear equations. *J. Glob. Optim.* **59**(2), 503–526 (2014)